

E - Logbook

Student	Jerome Bawa
Module	CS3IP The Psychological Narrative Engine (PNE)
University	University of Reading
Department	Computer Science
Academic Year	2025–2026

Introduction

This logbook documents the development of the Psychological Narrative Engine (PNE.) It is structured around the project’s Git commit history, grouped into sprint entries. Each entry summarises the activities completed, links those activities to the project’s objectives, and concludes with a reflection on what was learned and what would be done differently.

Entry 1 — Project Foundations

16 November 2025

Objective: Establish the project repository and define the core data model that all future pipeline stages will build on.

Activity	Detail
Repository initialised	Initial commit created. Basic project structure established with root directory and module stubs.
PNE_Models.py v1	Core data classes defined: CognitivePersonalityModel (self_esteem, locus_of_control, cog_flexibility), SocialPersonalityModel (assertion, empathy, ideology), WorldPerceptionModel (history, relation, known events), and NPCModel wrapping all three.
Pipeline + LLM import	pipeline.py started. qwen2.5:3b imported via Ollama for the first time — establishing that local LLM inference was feasible on available hardware (GTX 1650 Super, 4 GB VRAM).

Reflection	
WWW What Went Well	Good foundational design of NPC architecture and. Scalable modular system conceptualised.
EBI Even Better If	More consideration for World Perception Model in its unique design.

Entry 2 — Pipeline & Ollama Integration

17 – 22 November 2025

Objective: Wire Ollama into the pipeline and introduce a first outcome model so that player choices have mechanical consequences.

Activity	Detail
Ollama connected	qwen2.5:3b wired into pipeline.py to generate NPC dialogue. First successful end-to-end test — a player choice produced a real NPC response for the first time.
Outcome model added	Outcome type system added to pipeline.py: attribute changes, relation shifts, and narrative transitions represented as structured data. Bug in outcome data parsing found and fixed same day.
Coherence gap noted	Early testing revealed that without min/max response anchors, Ollama output was inconsistent in tone and sometimes off character. Logged as a future fix.

Reflection	
WWW What Went Well	Ollama integrated seamlessly. Outcome type system integrated where it was failed to be conceptualised in entry 1.
EBI Even Better If	Better, more consolidated decision-making made with narrative transitions.

Entry 3 — Full BDI Pipeline Established

3 – 4 December 2025

Objective: Complete the BDI processing chain end-to-end and reach a first feature-complete narrative simulator.

Activity	Detail
Full pipeline wired	pipeline.py expanded into a complete BDI chain: cognitive interpreter → desire formation → socialisation filter → outcome selector. Conversation simulation end-to-end became possible for the first time.
Narrative Simulator v1	narrative_engine.py declared feature-complete as a single-file template. ConversationSimulator.py deprecated. Capabilities: JSON scenario

Activity	Detail
	loading, BDI pipeline per turn, Ollama NPC response, terminal outcome detection.

Reflection	
WWW What Went Well	BDI chain fully established with few early console successes on a demo version.
EBI Even Better If	Automatic output logging should have been established here.

Entry 4 — LLM Comparison & Model Refactor

16 January 2026

Objective: Evaluate alternative local models and clean up the data model layer.

Activity	Detail
LLM switch to phi3:mini	Switched default model from qwen2.5:3b to phi3:mini for comparative evaluation. Quality and latency tested against the same scenarios. Later commits reverted to qwen2.5:3b after quality comparison favoured it.
Wildcards renamed	Wildcard system renamed to Prompt-Modifiers for clarity. The system injects Ollama sampling parameters (temperature, penalty, token limits) for rare emotional states. Behaviour unchanged.
PNE_Models.py refactor	Data model classes cleaned up. External config file loading added for wildcard/modifier definitions, making personality configurations editable without code changes.

Reflection	
WWW What Went Well	Refactoring made every entry from this point on feel easier to complete. Searching, debugging and maintenance was far quicker.
EBI Even Better If	Too much experimentation this entry; too many failures and too much time wasted here. Phi3: mini was inconsequential in the end.

Entry 5 — Multi-NPC Support & Package Restructure

27 January – 5 February 2026

Objective: Make the engine capable of multi-NPC scenarios and refactor the codebase into a maintainable package structure.

Activity	Detail
Multi-NPC narrative phase	Work began on how conversations unfold over multiple turns. Multi-NPC support, node/choice tree structure for scenario JSON, and per-NPC conversation state management added to narrative_engine.py.
Outcome-matching transitions	Transition logic rewritten to trigger on interaction_outcome from the BDI pipeline rather than on LLM dialogue content. Deterministic, reliable, and independent of what the model says. Confirmed with a full Krakk test conversation.
Package restructure	narrative_engine.py split into a proper Python package: engine.py (orchestrator), session.py (state), cli.py (terminal), scenario_loader.py, transition_resolver.py, choice_filter.py, dialogue_coherence.py. Old files moved to deprecated/.
Scenario JSON overhaul	Nodes now explicitly reference active NPCs. Multi-NPC scenarios no longer depend on an implicit single-NPC assumption.
Auto-generated docs	pdoc used to generate HTML API documentation for pne and narrative_engine packages.

Reflection	
WWW What Went Well	Good maintenance in package structure and doc generation. Transition logic expanded imperfectly but had massive influence on the engine..
EBI Even Better If	
	Multi – NPC support eventually abandoned.

Entry 6 — FSM, Judgement Tracker & Difficulty System

2 – 4 March 2026

Objective: Replace the naive turn-counter routing with a proper stateful FSM, introduce the judgement score, and add difficulty configuration.

Activity	Detail
Intention registry	Library of named intention templates added (e.g. “Carefully Question Motives”, “Propose Alliance”). Socialisation layer now scores against

Activity	Detail
	these rather than generating free-form text. FSM routing becomes deterministic.
World context in prompts	NPCs now receive a world-context snapshot (factions, locations, history) in their Ollama prompt. Previously they had only personal history and current conversation.
Pre-FSM checkpoint	Stable snapshot taken before the FSM rewrite. Scenario JSON restructured so nodes reference active NPCs directly. Multi-turn test conversations (Amourie Othella, Moses) recorded to inform FSM design.
Full FSM routing	Scenario nodes gain transition conditions: turn count, relation thresholds, and intention-keyword gates. Engine picks first matching transition; stays on current node if none match. Recovery mode added — failed rolls queue a second-chance choice set.
success_pct added	Each choice now displays an analytical pre-roll success probability (0–100%) before the player commits. Calculated from skill, NPC resistance, and current relationship.
Judgement tracker	Judgement score (0–100, starting at 50) introduced per NPC. Replaces raw player_relation float as the primary FSM routing driver. Risk multiplier applied when pre-roll odds were against the player.
NE.py entry point	Clean top-level launcher created. Old monolithic narrative_engine.py removed.
Difficulty system	SIMPLE (+15%), STANDARD (0%), STRICT (–15%) difficulty presets added as additive dice bias on top of relationship bonus.
Failed-choice pruning	Choices that fail both the main roll and recovery roll are permanently hidden from future turns.
Test logs saved	Conversation snapshots for Troy, Krakk, Amourie Othella, and Morisson Moses saved as .json for regression testing and dissertation evidence.

Reflection	
WWW What Went Well	Massive number of overhauls completed. Most effort within an entry, especially within a few days. Most of the engine's core successful claims refined or explored purposefully here.
EBI Even Better If	More test logs were saved.

Entry 7 — CognitiveThoughtMatcher & REST API

9 March 2026

Objective: Replace the LLM-based cognitive layer with a deterministic template matcher, and expose the engine as a network API ready for game-engine integration.

Activity	Detail
CognitiveThoughtMatcher	cognitive_thought_matcher.py introduced with 810 scored thought templates in cognitive_thoughts.json. Template selection is a weighted scoring algorithm (≈ 0 ms). Replaces LLM call #1, which was taking 4–8 seconds per turn. cognitive.py and desire.py substantially rewritten.
REST API layer	Full Models/api/ package: FastAPI server (main.py), in-memory session store (session_store.py), Pydantic schemas (schema.py), WebSocket handler (ws_handler.py), NPC state updater (npc_updater.py). Ollama token streaming bridged through WebSockets.
Unity client library	PNEClient.cs, PNEDialogueUI.cs, PNETypes.cs and a full Unity integration guide (HTML + Markdown) added under docs/unity/.
Godot client script	docs/api_client_godot.gd provides Godot 4 GDScript integration.
Install scripts	install.bat, install.sh, and a top-level requirements.txt added. Engine setup reduced to a single command on Windows or Unix.
Troy NPC overhaul	troy.json significantly revised with updated BDI attributes and scenario hooks aligned to the test corpus.

Reflection	
WWW What Went Well	This is where the PNE was technically feasible for the first time, could reach an end-user!
EBI Even Better If	Clearer design consideration for exemplary Unity script at the very least. Minor considerations for the GDScript version at least.

Entry 8 — Codebase Documentation Pass

12 March 2026

Objective: Bring documentation up to submission standard across all core modules.

Activity	Detail
Comprehensive docstrings	Full docstrings added to all public classes and methods across engine.py, dialogue_coherence.py, skill_check.py, processor.py, desire.py, player_input.py, intention_registry.py, social.py, schema.py. Module-level summaries, attribute tables, argument/return descriptions.
Module headers rewritten	Top-of-file blurbs for engine.py, dialogue_coherence.py, and skill_check.py replaced with structured overviews: role in pipeline, exported classes, interaction patterns.

Activity	Detail
Schema contracts documented	Every Pydantic model (CreateSessionRequest, Choiceltem, ChoicesResponse, SaveResponse) documents field purpose, valid values, and which API endpoint uses it.
Constants documented	DIFFICULTY_ADJ, LANGUAGE_ART_TO_SKILL, SKILL_MODIFIERS tables in SkillCheckSystem given explanatory comments. DiceCheckResult and SkillCheckResult field comments standardised.
Architecture PDF updated	Pipeline Design PDF revised to reflect current REST API layer and BDI pipeline structure.

Reflection	
WWW What Went Well	Good documentation for final report! Helpful for writing chapters.
EBI Even Better If	More graphs! Had to do them closer to deadline than desired.

Sprint 9 — Final Submission Preparation

10 April 2026

Objective: Fix outstanding script issues and create the final dissertation graph tables for submission.

Activity	Detail
run_engine.sh fixed	Launch script was pointing to non-existent narrative_engine.py. Corrected to NE.py. Confirmed engine launches correctly from the root directory.
config.py centralised	OLLAMA_MODEL confirmed as qwen2.5:3b and sourced consistently by ollama_integration.py across the entire system. No hard-coded model strings remain.
Dissertation tables created	Three reference tables added to Dissertation/Literature Material/Drafts/Graphs/: 5.1.1.md (pipeline stage overview, 7 stages), 5.1.4 Table.md (NPC data structure, Amourie Othella worked example), 5.2.2.md (door_guard_night scenario routing log, turn-by-turn).

Overall Project Reflection

This section provides a summary reflection on the project as a whole, covering the most significant decisions, the gap between what was planned and what was delivered, and the main learning outcomes.

Reflection

WWW What Went Well

Good mixture of experimentation, iteration, testing and documentation. Clear consideration of modularity with scalability in mind. Should be somewhat feasible expanding in the future.

EBI Even Better If

No clear testing automation system. Neither for storing test information either.